ReRu Redis Compatibility Implementation Plan

Team execution plan | 2 October 2026 | Repository baseline b27be36

Purpose and completion standard

ReRu aims to become a Redis-compatible server in Rust that other developers can use. The team should first deliver a dependable standalone server, then complete the pinned Redis command and server-feature inventory. The current repository is a prototype, so release claims must follow measured compatibility and operational tests.

The repository roadmap pins Redis 8.10.2 as the behavior reference. Keep that target fixed for the first compatibility matrix; change it only through an explicit version decision. Treat a feature as complete only when normal cases, errors, binary payloads, expiry, concurrency, and restart behavior match the pinned server where applicable.

Verified baseline on 2 October 2026

Area	Implemented now	Important limit
Protocol and execution	RESP2 array requests; concurrent TCP clients; shared in-memory store.	One OS thread per client; no RESP3, client limit, or backpressure policy.
Data and commands	Strings, lists, streams; 19 public command names, including MULTI, EXEC and DISCARD.	Hashes, sets, sorted sets, bitmaps, geo, many keyspace commands and WATCH remain.
Durability	Optional --snapshot saves a ReRu binary snapshot after writes and loads it at startup.	Not Redis RDB or AOF; no append log or crash-recovery matrix.
Replication	Read-only ReRu replica receives an initial snapshot and later full snapshots.	Not Redis PSYNC/RDB, offsets, ACK/WAIT, or failover.

Evidence: 1,544 Rust source lines and 816 Rust test lines are tracked. The local suite passes 35 tests (2 unit and 33 integration). These counts describe the current repository; they are not a percentage of Redis compatibility.

Release boundary

- Publish a machine-readable command and feature matrix for the pinned Redis version, with implemented, partial, unsupported, and verified states.
- Do not call ReRu a full Redis replacement until the target matrix, interoperability suite, recovery suite, and release gates on page 4 all pass.

Target architecture

Keep one deterministic order for writes and transaction execution while networking, disk I/O, and replica delivery operate asynchronously. Separate protocol parsing from command semantics so a command can be tested without opening a socket.

Boundary	Responsibility and interface
Network and RESP	Async TCP acceptor, incremental RESP2/RESP3 parser, binary-safe frames, request and output limits, cancellation and backpressure. Emits typed requests.
Command registry	One metadata record per command: arity, flags, ACL category, key positions and version. Validates arguments before dispatch; returns typed replies.
Execution engine	Single logical write sequencer owns keyspace state, expiry index and key revisions. Runs MULTI/EXEC atomically; WATCH compares revisions. Blocking operations register waiters rather than hold a lock.
Data model	Modular values for strings, lists, hashes, sets, sorted sets and streams; bitmap and geo semantics use the proper underlying encodings. Shared key metadata carries TTL and version.
Durability	Ordered mutation stream feeds AOF; point-in-time RDB writer and loader provide Redis-compatible files. Recovery replays the selected format and checks corruption/expiry rules.
Replication	Replication ID, byte offset and bounded backlog support PSYNC. Full sync transfers RDB, then ordered writes; replicas ACK offsets and support WAIT.
Operations	Configuration, ACL/authentication, INFO/metrics, memory policy, graceful shutdown, logging and diagnostic commands are explicit modules.

Critical invariants

- Binary keys and values remain byte arrays end to end; no UTF-8 assumption enters the execution path.
- A write becomes visible, durable under the configured policy, and replicable in the same global order. EXEC is indivisible to other clients.
- Expiry uses a monotonic clock for live behavior and an absolute wall-clock deadline for persistence and replication.
- A slow client or replica cannot grow memory without a configured bound or block unrelated command execution.

Migration from the current code

The present main.rs combines TCP handling, command dispatch, transactions and replication; SharedStore uses Mutex and Condvar. Extract modules behind tests first, preserve current RESP replies, then replace global locking with the sequencer. Keep the existing snapshot reader only as a migration path until Redis-compatible persistence is validated.

Delivery milestones and dependencies

Use two-week iterations and merge only tested vertical slices. The exit gate for each milestone is a runnable server, passing local tests and a recorded differential result against the pinned Redis build. Calendar dates follow a command inventory and team capacity review.

Milestone	Scope	Exit gate
M0 Inventory	Catalog every target command, option and server feature; classify current behavior and licensing approach.	Versioned matrix, owner map and golden Redis fixtures approved.
M1 Core engine	Extract parser/registry/executor; add RESP3 negotiation, limits, expiry index, keyspace commands and typed errors.	No protocol regression; concurrency and resource-limit tests pass.
M2 Data types	Finish lists/streams; add hashes, sets, sorted sets, bitmaps and geo command families.	Each family has normal, edge, binary, wrong-type and expiry differential tests.
M3 Coordination	WATCH/UNWATCH, full transaction semantics, pub/sub, blocking waiters and scripting decisions.	Atomicity, queue-error, runtime-error and waiter-order tests pass.
M4 Persistence	Redis RDB read/write, AOF append/rewrite, fsync policies, recovery and file migration.	Restart, torn-write, expiry and corruption tests pass with documented durability modes.
M5 Replication	PSYNC, replication IDs and offsets, full/partial sync, backlog, ACK and WAIT.	Interoperate with a pinned Redis primary/replica in supported topologies; reconnect tests pass.
M6 Operations	ACL/auth, configuration, memory limits/eviction, observability, packaging and security review; cluster only after standalone stability.	Release candidate passes compatibility, load, recovery and operational gates.

Backlog rule

A command ticket names the Redis version, exact syntax and options, state transitions, reply bytes, error cases, persistence effect, replication effect and test fixture. Mark it verified only after differential testing; merely accepting its name is partial support.

Work that can proceed in parallel

Once the command registry and value interfaces stabilize, data-type families can be owned independently. Persistence and replication can build in parallel on the same ordered mutation stream, but must integrate before their exit gates.

Quality and release gates

Run correctness checks before performance tuning. A pinned Redis executable acts as the behavior oracle for supported commands; ReRu and Redis receive identical requests and their protocol replies and resulting state are compared. Track known deviations explicitly.

Test layer	Required coverage
Protocol	Fragmented/pipelined requests, malformed frames, binary data, RESP2/3 replies, oversized input and slow readers.
Behavior	Golden cases and property tests per command, including bounds, overflow, wrong type, missing keys, TTL and transaction errors.
Concurrency	Concurrent clients, atomic EXEC, WATCH races, blocked lists/streams, pub/sub ordering and cancellation.
Durability	Kill/restart at write boundaries; RDB/AOF import/export, truncated files, checksum failures, fsync settings and expired keys after downtime.
Replication	Initial sync, partial resync, disconnection, slow replica, offset/ACK accounting and WAIT semantics.
Load and security	Latency/throughput at stated dataset sizes, memory bounds, fuzzed parsers, ACL checks and denial-of-service cases.

Continuous integration

- On every pull request: rustfmt, Clippy with warnings denied, unit/integration tests, protocol fixtures and the changed-command differential suite on Windows and Linux.
- Nightly: full differential matrix, fuzz corpus, crash/restart suite, replica fault injection and benchmark trend comparison.
- Before release: security review, license review, reproducible builds, upgrade/rollback rehearsal, documentation and client-library interoperability checks.

Definition of a usable release

Publish an exact supported-command list and configuration subset, not a blanket compatibility claim. State persistence guarantees, replication limits, tested operating systems, memory limits and benchmark conditions. A general-purpose Redis replacement requires the complete target matrix and the gates above, with no unclassified commands.

Team execution and decisions

Owner	Accountability
Technical lead	Own compatibility target, architecture decisions, command matrix, integration sequencing and release claims.
Engine and protocol	Own RESP, command registry, execution order, expiry, transactions and resource limits.
Data and commands	Own value modules and command families; supply differential fixtures for each merged slice.
Durability and replication	Own ordered mutation stream, RDB/AOF, PSYNC, crash tests and replica fault tests.
Quality and release	Own CI, Redis oracle harness, benchmarks, packaging, documentation and security gates.

First two iterations

Iteration 1: freeze the pinned Redis build, generate the command/feature inventory, set up a real Redis differential harness, and record failures for the 19 current commands. Iteration 2: extract the command registry and executor, add key revisions for WATCH, and introduce ordered mutation events used by durability and replication.

Risks to resolve early

Risk	Mitigation
Unbounded scope	Freeze a versioned matrix and prioritize standalone parity before clustering or extensions.
Durability/order mismatch	Use one mutation order for execution, AOF and replica offsets; test crashes at boundaries.
Source license obligations	Implement from public behavior specifications; obtain legal review before translating Redis 8 source code.
Prototype architecture	Refactor behind existing tests before adding many command families or claiming production readiness.

Reference specification

Redis protocol: https://redis.io/docs/latest/develop/reference/protocol-spec/

Redis persistence: https://redis.io/docs/latest/management/persistence/

Redis replication: https://redis.io/docs/latest/operate/oss_and_stack/management/replication/

Redis source and license: https://github.com/redis/redis and https://github.com/redis/redis/blob/unstable/LICENSE.txt